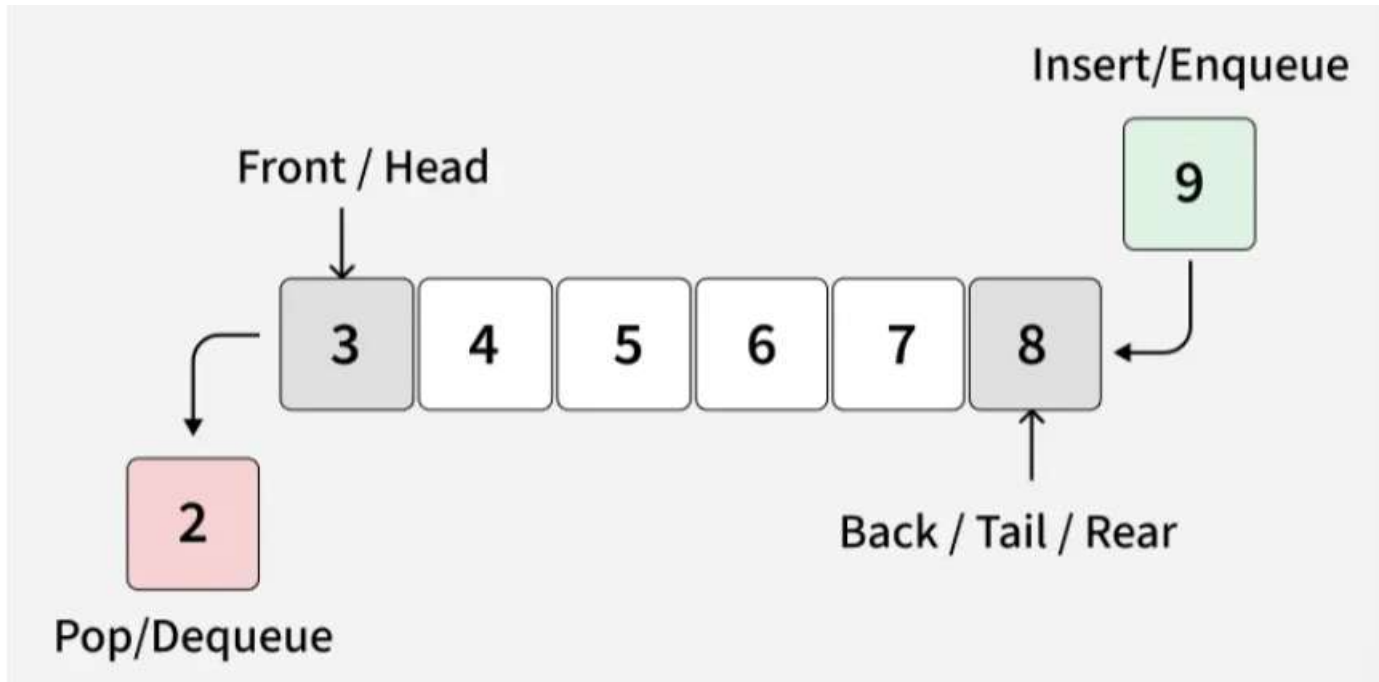


Queue

Tuesday, May 5, 2026 11:07 AM

It is a linear data structure that stores items in a FIFO manner. The item that is added first will be removed first.



We can implement it in different ways:

- Using list: Inefficient

```
q = []
q.append('a')
q.append('b')
q.append('c')

print("Initial queue:", q)

print("Elements dequeued from queue:")
print(q.pop(0))
print(q.pop(0))
print(q.pop(0))

print("Queue after removing elements:", q)
```

- Using collections.deque: Efficient because both append and popleft are $O(1)$ time.

```
from collections import deque
q = deque()

q.append('a')
q.append('b')
q.append('c')
print("Initial queue:", q)

print("Elements dequeued from the queue:")
print(q.popleft())
print(q.popleft())
print(q.popleft())
```

```
print("Queue after removing elements:", q)
```

- Using queue.Queue: efficient and thread-safe. Key methods are:
 - o Put/ put_nowait
 - o Get/ get_nowait
 - o Empty
 - o Full
 - o Qsize

```
from queue import Queue  
q = Queue(maxsize=3)  
print("Initial size:", q.qsize())
```

```
q.put('a')  
q.put('b')  
q.put('c')  
print("Is full:", q.full())
```

```
print("Elements dequeued from the queue:")  
print(q.get())  
print(q.get())  
print(q.get())  
print("Is empty:", q.empty())
```

```
q.put(1)  
print("Is empty:", q.empty())  
print("Is full:", q.full())
```